assignment1

Jingliang Guo

netid: nxt1510

1. Shuttle: model, solution and checks

State s is the number waiting (0-200); action a is wait (0) or dispatch (1). Capacity is 15. Independent arrivals are uniform over {1,2,3,4,5}.

```
remaining = max(s - 15*a, 0)
cost = 100*a + 2*remaining
next_state = min(200, remaining + arrivals)
gamma = 0.95
```

Waiting cost is charged after dispatch and before arrivals. Excess arrivals leave without penalty. An empty dispatch costs 100. The finite-horizon terminal value is zero.

```
Q_t(s,a) = cost(s,a) + gamma * mean(V_(t+1)[next_state])
V_t(s) = min_a Q_t(s,a)
```

Enumeration: check both actions and all five arrivals at every state and time, working backward from T=500. **Value iteration:** apply Bellman updates from V=0 until the largest change is below 1e-6. **Policy iteration:** alternate policy evaluation (tolerance 1e-8) and improvement until no action changes.

Enumeration here means exhaustive state-action-outcome evaluation with reused future values. Ties wait in enumeration/value iteration; policy iteration keeps its current action.

Validation / edge case	Expected result	Observed
gamma=0	Always wait; V(s)=2s	201/201 actions correct; value error 0
Empty / partial / full shuttle	Dispatch costs 100 at s=0 or 15; 102 at s=16	7 hand-calculated cases pass
Queue cap	From 198 remaining, P(next=200)=4/5	3 expectation checks pass
Always wait at the cap	V(200)=400/(1-0.95)=8000	Largest analytic error 1.89e-7
Compare all three methods	Same policy; finite/infinite values close	201/201 actions agree; value gap 1.81e-5

The analytic cases give independent answers; agreement between implementations alone could miss a shared error.

2. Shuttle: results

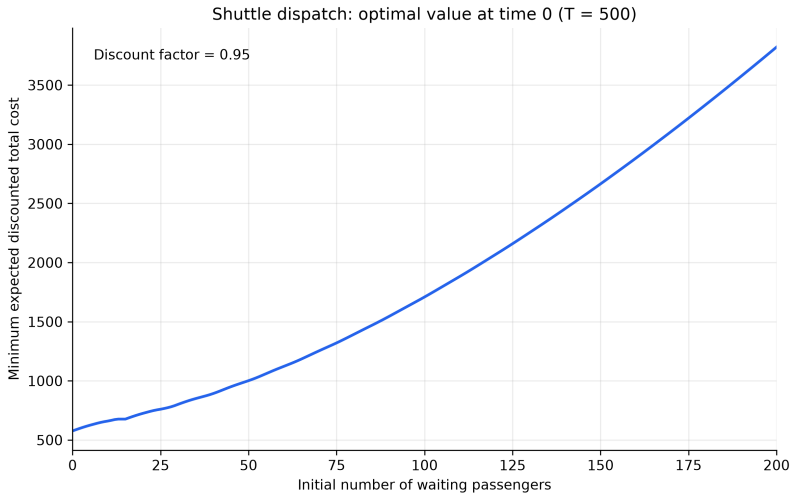


Figure 1. Enumeration: minimum expected discounted cost at time 0, T=500.

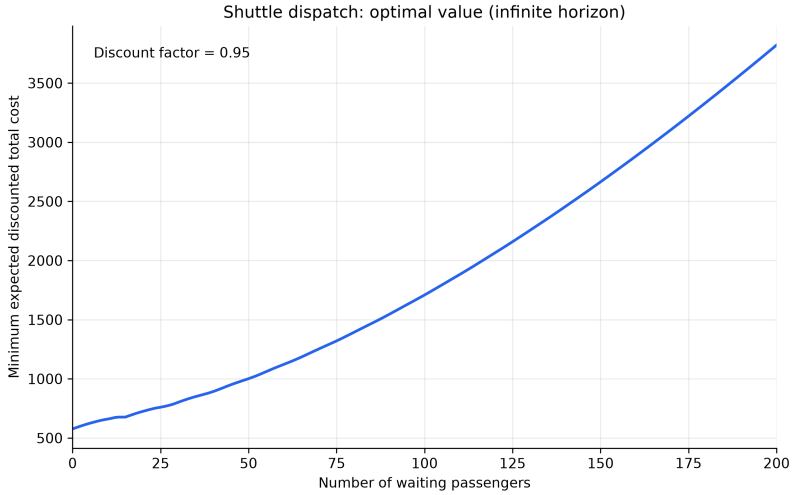


Figure 2. Infinite-horizon value iteration: optimal value versus queue size.

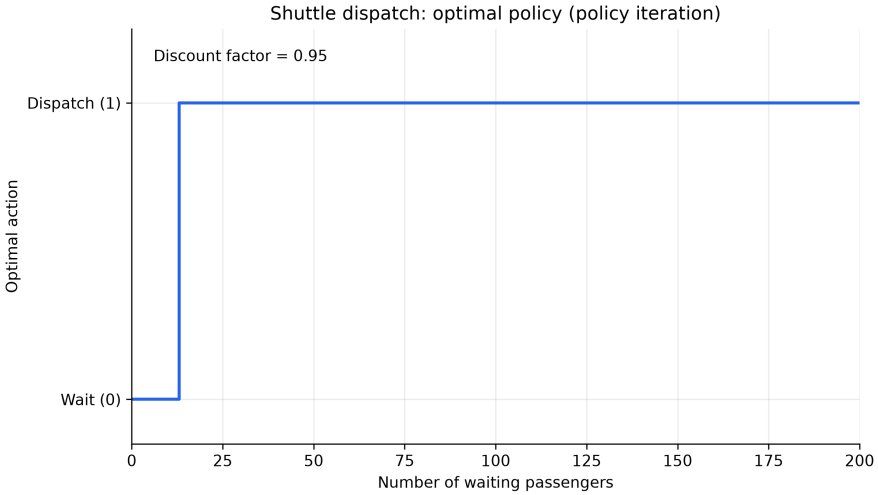


Figure 3. Policy iteration: wait below 13 passengers; dispatch at 13 or more.

Larger queues cost more. The value curves nearly overlap at gamma=0.95. Value iteration takes 340 sweeps; policy iteration takes 6 rounds.

3. Policy gradient: original method

REINFORCE uses one ReLU hidden layer and two action logits. Actions are sampled in training and testing. The original four runs update Adam once per episode.

Setting	CartPole-v1	ALE/Pong-v5
Input / actions	4 state values; actions 0/1	6400 frame differences; actions 2/3
Network	4 -> 32 -> 2	6400 -> 128 -> 2
gamma / learning rate	0.95 / 0.01	0.99 / 0.001
Training episodes per setting	2000	5000
Initialization	PyTorch default	He hidden layer; output gain 0.01; zero biases

Pong uses the supplied crop, downsampling and binary preprocessing, followed by consecutive-frame subtraction. Previous-frame memory resets each episode. The v5 defaults remain: frameskip 4 and sticky-action probability 0.25.

I reduced the Pong learning rate to 0.001 for stability. In the 0.01 trial, 72.2% of probe states had an action probability above 99% after 100 training games, while the 20-game validation mean remained -20.90.

```
G[T] = 0
G[t] = r[t] + gamma*G[t+1]
loss = -sum_t gamma^t * (G[t]-b) * log pi(a[t]|s[t])
```

Returns continue across all Pong points. Both termination and truncation end an episode; there is no value bootstrap. Gradients accumulate in 256-state chunks before a single update.

Baseline comparison

Part 1 uses b=0. Part 2 uses a scalar historical EMA, computed before the current episode. Each game uses matched starting weights, seeds and budgets.

```
# After the policy update:
reference = sum_t(gamma^t*G[t]) / sum_t(gamma^t)
b = 0.9*b + 0.1*reference
```

Each original final policy is frozen and tested for 500 games. Scores are undiscounted reward sums; standard deviations use ddof=0. These tests are separate from the follow-up training results on page 6.

Game	Baseline	Test mean	Test SD
CartPole	none	123.094	36.230
CartPole	ema	369.722	65.631
Pong	none	-18.374	2.743
Pong	ema	-18.786	2.709

4. CartPole: original results

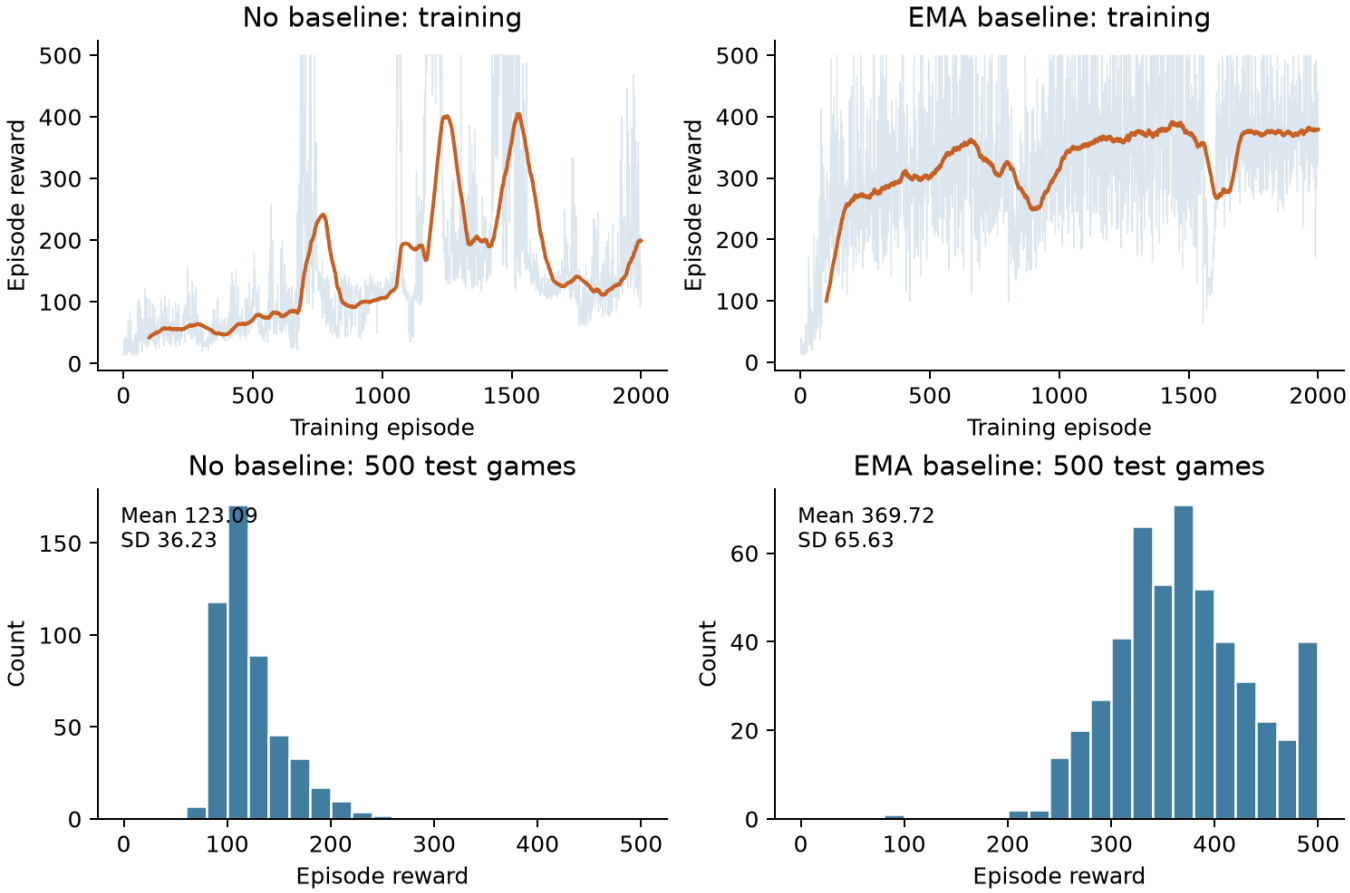


Figure 4. Original CartPole runs. Top: episode rewards (blue) and trailing 100-episode means (orange). Bottom: 500-game frozen-policy test histograms.

The EMA policy scores 369.722 versus 123.094 without a baseline, a gain of 246.628 in this pair of runs. Both learning curves fluctuate, so training improvement is not monotonic.

This is one training seed per setting. The comparison supports an improvement for these two trained models, rather than a general ranking across retraining.

5. Pong: original results

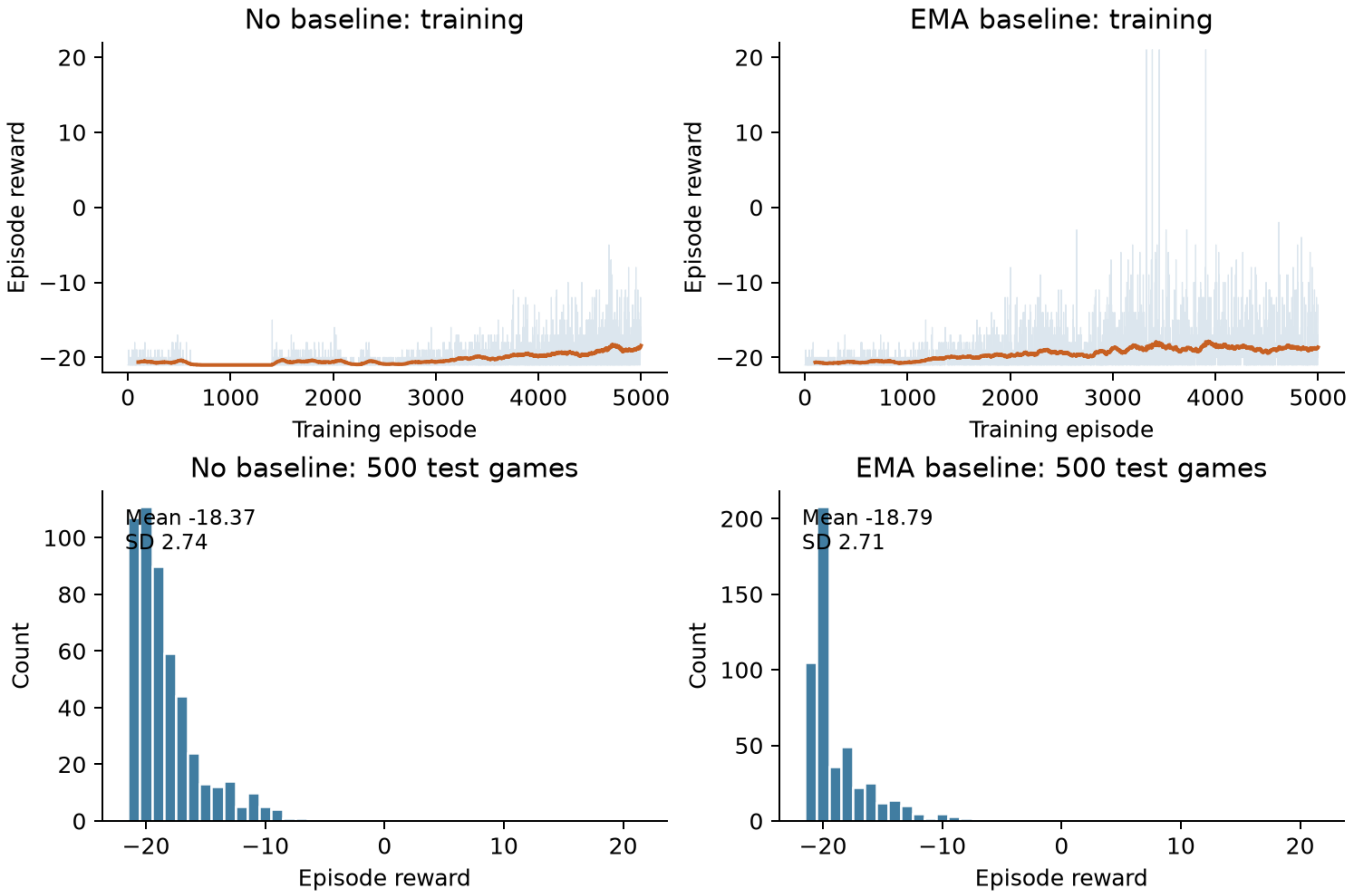


Figure 5. Original Pong runs. Top: episode rewards and trailing 100-episode means. Bottom: 500-game frozen-policy test histograms.

Original setting	Training episodes	Test games	Test mean	Test win rate
No baseline	5000	500	-18.374	0%
EMA baseline	5000	500	-18.786	0%

Both models improve late but still lose every test game. EMA reduces measured gradient variance (page 7), yet it does not improve the final whole-game score in these runs.

Learning-rate, width, update-frequency and initialization trials did not produce a comparable improvement within their tested budgets. I therefore changed how returns enter the update in the separate follow-up experiment.

6. Pong: standardized returns

I standardize each episode's advantages to provide a relative performance reference and a consistent scale. I also remove the outer gamma^t weight, so late-game updates are not additionally suppressed. Full-game returns still use gamma=0.99; returns are **not reset at scored points**.

```
A[t] = (G[t]-b - mean(G-b)) / (std(G-b) + 1e-8)
loss = -sum_t A[t] * log pi(a[t]|s[t])
```

Adam uses lr=0.001 and averages gradients over 10 games per update. The network remains 6400 -> 128 -> 2, with output gain 0.1 as in the preceding trial. The constant EMA cancels under centering, so this is not evidence of an EMA benefit.

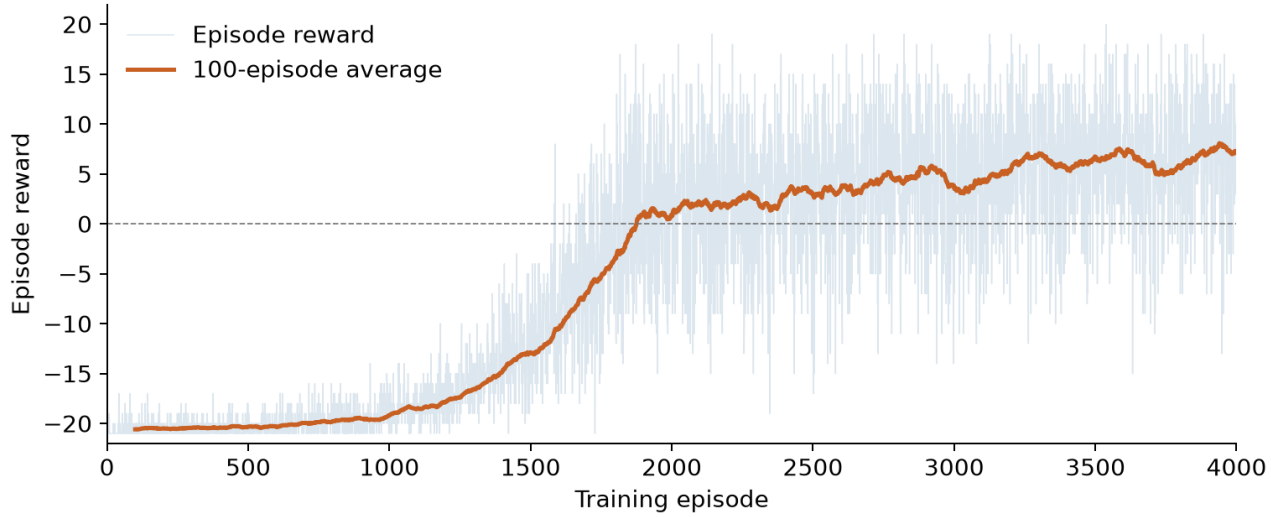


Figure 6. Follow-up: episode rewards and the trailing 100-episode mean, episodes 1-4000.

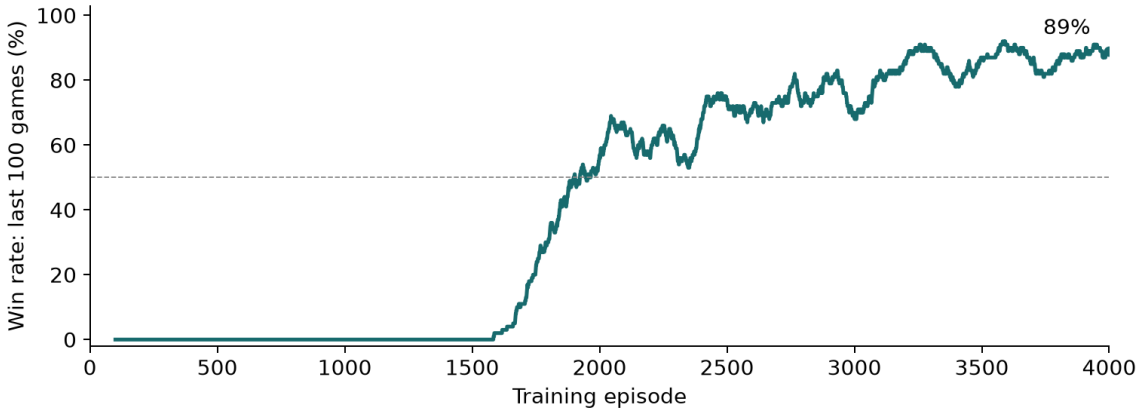


Figure 7. Training win rate over the most recent 100 games; a win has positive episode reward.

Episode	Last-100 mean	Last-100 win rate
100	-20.59	0%
1000	-19.18	0%
2000	+0.77	56%
4000	+7.20	89%

The combined changes improve training from -20.59 and 0% wins to +7.20 and 89% wins. Standardization and outer-weight removal were changed together; this run does not isolate their effects.

Training curves show the first 4000 games. The final checkpoint was saved at 4160; its independent 500-game test is on page 7. This follow-up changes the estimator; the original discounted-objective comparison is retained.

7. Final test and validation

The frozen 4160-game checkpoint scores **+5.938 (SD 5.859)** on 500 new games, winning **425/500 (85.0%)**. Actions are sampled from the policy; no parameters are updated.

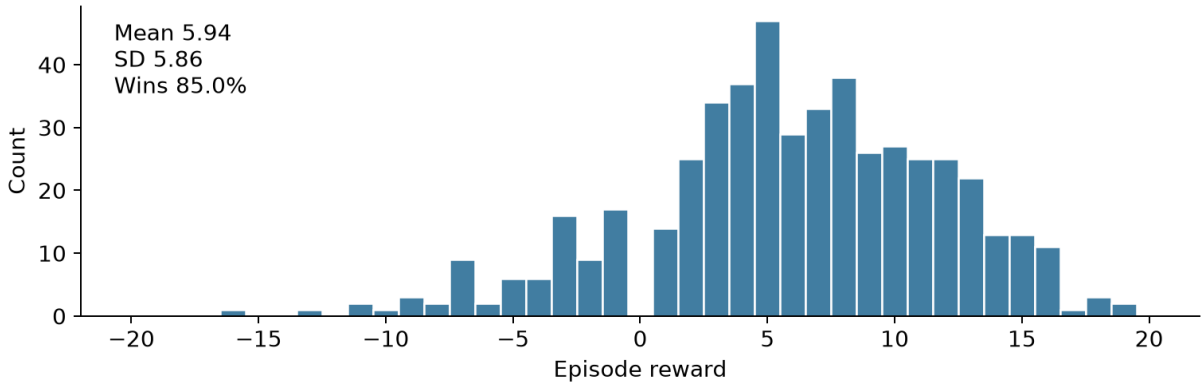


Figure 8. Independent 500-game test of the final standardized-return Pong model.

Original-method check	Expected behavior / procedure	Observed
Exact gradient	Enumerate all four two-step trajectories; compare REINFORCE with direct differentiation, b=0 and b=0.8	Max errors 8.94e-8 and 1.49e-8
Full vs chunked gradient	Same 887-step Pong trajectory; small descent step should lower its fixed-sample loss	Relative error 8.46e-8; loss decreases
Seeded Pong replay	Fixed policy and both environment/action seeds should reproduce a trajectory	Identical 946-step traces; score -21

Baseline variance check

With a frozen policy, estimate b from 20 separate pilot games. Compare gradients on paired fresh trajectories using the sum of component sample variances.

Game	Paired games	No baseline	EMA	EMA / none
CartPole	40	891.86	151.81	0.1702
Pong	20	609.03	107.61	0.1767

The variance reduction is about 83% for CartPole and 82% for Pong at these policies; the Pong check uses a default-initialized policy. This does not guarantee higher scores. All trained settings use one seed, and the follow-up does not isolate standardization from outer-weight removal.

Edge cases: preprocessing preserves raw frames; frame memory resets each game; both done flags are handled. Log probabilities use logits and non-finite losses are rejected. The saved replay starts with 10 zero rewards before any point is scored.

With three more months

Repeat matched-budget ablations across at least five seeds; test value baselines and gradient variance at trained checkpoints. Use fresh test seeds for selected models. For the shuttle, vary arrivals, capacity and cap rules, checking deterministic cases against exact solutions.

8. Code and reproduction

GitHub: jingliangguo2027 / mlds-assignment-1
https://github.com/jingliangguo2027/mlds-assignment-1

From hw1, install a suitable PyTorch build, then:

```
python -m pip install -r requirements.txt
python main.py
python value_iteration.py
python policy_iteration/main.py
python policy_iteration/validate.py --experiment all
```

Original experiments

From policy_gradient, replace BASELINE with none, then ema. Use new output folders.

```
python train.py --game cartpole --baseline BASELINE \
  --episodes 2000 --eval-episodes 500 --lr 0.01 \
  --output new_cartpole_BASELINE
CUDA_VISIBLE_DEVICES=0 python train.py --game pong \
  --baseline BASELINE --episodes 5000 --eval-episodes 500 \
  --hidden 128 --initialization he_small_output --lr 0.001 \
  --device cuda --output new_pong_BASELINE
```

Pong follow-up

```
CUDA_VISIBLE_DEVICES=0 python pong_standardized.py \
  --episodes 4160 --device cuda --output new_pong_standardized
```

The saved training curves use the first 4000 games; the final test uses the checkpoint at 4160. The command above reproduces the final endpoint; evaluate its checkpoint with:

```
CUDA_VISIBLE_DEVICES=0 python pong_standardized.py \
  --evaluate-checkpoint new_pong_standardized/checkpoint.pt \
  --eval-episodes 500 --vector-envs 8 --device cuda \
  --output new_pong_test
```

Redraw recorded figures with --plot-only --output results/pong_standardized.

Validation and report

```
# From policy_gradient:
python validate.py
CUDA_VISIBLE_DEVICES=0 python validate.py --extra pong-variance --device cuda
CUDA_VISIBLE_DEVICES=0 python validate.py --extra chunk --device cuda
# From hw1:
python build_report.py
```

Default validation checks exact gradients, Pong preprocessing/replay and CartPole gradient variance. Extra modes check Pong variance and chunked gradients. They print fresh results without overwriting saved evidence.

Scores and settings: policy_gradient/results/. Validation and tuning evidence: policy_gradient/evidence.json. New curves use results/pong_standardized/metrics.json; the new test is in evaluation.json. Original test seeds: 1000000-1000499; tuning: 200000-200049; new test: 9000000-9000499, with independent per-game action RNGs.

Original Pong runners use action seed 10000+episode and environment seed episode. PyTorch 2.14.1 (+cu126 for Pong), Gymnasium 1.4.0, ALE 0.12.1; Pong uses one RTX A6000. Exact trajectories can vary across runtime versions. Historical scripts remain in commit e8ec206.